

2.1 Component Basics

Part of Components Fundamentals Module - Duration: 10-12 hours

1. What is a Component?

1.1 Definition

A component is the fundamental building block of an Angular application. It controls a portion of the user interface (UI) and defines how that portion should look and behave. Every Angular application has at least one component - the root component - and is essentially a tree of components.

Think of components as reusable, self-contained units that encapsulate:

- **Structure:** The HTML template that defines the view
- **Logic:** The TypeScript class that handles data and behavior
- **Style:** The CSS that defines the appearance

```
// Simple analogy: Building a house
House (Application)
├── Kitchen (Component)
│   ├── Structure: Layout and fixtures
│   ├── Logic: How appliances work
│   └── Style: Colors and design
├── Bedroom (Component)
│   ├── Structure: Bed, closet, windows
│   ├── Logic: Light switches, door locks
│   └── Style: Paint, curtains, flooring
└── Bathroom (Component)
    ├── Structure: Sink, shower, toilet
    ├── Logic: Plumbing, electrical
    └── Style: Tiles, fixtures, mirrors
```

Output:

Component Concept: Each room (component) is self-contained, has its own purpose, and can be designed independently. Similarly, Angular components are independent, reusable pieces that make up your application.

1.2 Why Components?

Components solve several critical problems in web development:

Problem	Component Solution
Code Duplication	Create once, reuse everywhere (e.g., button, header, card)
Maintenance	Update in one place, changes reflect everywhere
Organization	Break complex UI into manageable, logical pieces
Testing	Test each component independently in isolation
Team Collaboration	Different developers work on different components simultaneously
Scalability	Add new features without affecting existing code

2. Component Anatomy

2.1 The Three Parts of a Component

Every Angular component consists of three primary parts that work together:

2.1.1 TypeScript Class (Component Logic)

The TypeScript class contains the component's data (properties) and behavior (methods).

```
// user-profile.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-profile',
  templateUrl: './user-profile.component.html',
  styleUrls: ['./user-profile.component.css']
})
export class UserProfileComponent {
  // Properties (data)
  userName: string = 'John Doe';
  userAge: number = 28;
  isActive: boolean = true;

  // Methods (behavior)
  greetUser(): string {
    return `Hello, ${this.userName}!`;
  }

  incrementAge(): void {
    this.userAge++;
  }
}
```

Output:

TypeScript Class Contains:

- `userName` , `userAge` , `isActive` - Properties that store data
- `greetUser()` , `incrementAge()` - Methods that define behavior
- Properties and methods are accessible in the template using `{{ }}` or event binding

2.1.2 HTML Template (Component View)

The HTML template defines what the component displays to the user.

```
<!-- user-profile.component.html -->
<div class="profile-card">
  <h2>{{ userName }}</h2>
  <p>Age: {{ userAge }}</p>
  <p>Status: {{ isActive ? 'Active' : 'Inactive' }}</p>
```

```
<div class="greeting">
  {{ greetUser() }}
</div>

<button (click)="incrementAge()">Increase Age</button>
</div>
```

Output:

Browser Display:

John Doe

Age: 28

Status: Active

Hello, John Doe!

Increase Age

When button is clicked, age increases to 29, 30, 31...

2.1.3 CSS Styles (Component Appearance)

The CSS file defines how the component looks.

```
/* user-profile.component.css */
.profile-card {
  border: 2px solid #3f51b5;
  border-radius: 8px;
  padding: 20px;
  max-width: 400px;
  margin: 20px auto;
  background-color: #f5f5f5;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

h2 {
  color: #3f51b5;
  margin-top: 0;
```

```
}

.greeting {
  background-color: #e8eaf6;
  padding: 10px;
  border-radius: 4px;
  margin: 10px 0;
  font-weight: bold;
}

button {
  background-color: #3f51b5;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 4px;
  cursor: pointer;
  font-size: 14px;
}

button:hover {
  background-color: #303f9f;
}
```

Output:

Styled Output: The profile card now has a blue border, rounded corners, shadow effect, and the button has a hover effect. These styles apply **ONLY** to this component, not to other components in the application.

⚡ Important:

Style Encapsulation: By default, Angular uses view encapsulation, which means the CSS styles in `user-profile.component.css` apply only to the user-profile component and won't affect other components. This prevents style conflicts.

3. @Component Decorator and Metadata

3.1 What is a Decorator?

A decorator is a special function in TypeScript that adds metadata to a class, method, or property. In Angular, the `@Component` decorator transforms a plain TypeScript class into an Angular component by providing configuration metadata.

```
import { Component } from '@angular/core';

@Component({
  // Metadata configuration goes here
  selector: 'app-user-profile',
  templateUrl: './user-profile.component.html',
  styleUrls: ['./user-profile.component.css']
})
export class UserProfileComponent {
  // Component class code
}
```

Note:

The `@Component` decorator is placed directly above the class definition and contains a configuration object with various properties that tell Angular how to process and use this component.

3.2 Key Metadata Properties

3.2.1 selector

The `selector` defines the custom HTML tag that represents this component. When Angular encounters this tag in a template, it renders the component.

```
@Component({
  selector: 'app-user-profile', // Custom HTML tag
  // ...
})
export class UserProfileComponent { }
```

Usage in another component's template:

```
<!-- app.component.html -->
<div>
  <h1>My Application</h1>
  <app-user-profile></app-user-profile> <!-- Component rendered here -->
</div>
```

Output:

Result: When Angular sees `<app-user-profile>`, it replaces it with the component's template (the HTML from `user-profile.component.html`) and applies the component's logic and styles.

Selector Naming Conventions:

Convention	Example	Usage
Prefix	<code>app-user-profile</code>	Use app- prefix to avoid conflicts with native HTML elements and third-party libraries
Kebab-case	<code>app-user-profile</code>	Use lowercase with hyphens (not camelCase or PascalCase)
Descriptive	<code>app-user-profile</code>	Name should clearly indicate component purpose

3.2.2 template vs templateUrl

You can define the component's HTML either inline or in a separate file.

Option 1: Inline Template

```
@Component({
  selector: 'app-greeting',
  template: `
    <div class="greeting-box">
      <h3>{{ title }}</h3>
      <p>{{ message }}</p>
    </div>
  `,
  styles: [
    .greeting-box {
      border: 1px solid #ccc;
      padding: 10px;
    }
  ]
})
export class GreetingComponent {
  title = 'Welcome';
  message = 'Hello, Angular!';
}
```

Output:

When to use inline template:

- Template is very short (3-5 lines)
- Component is simple and self-contained
- Quick prototyping or demo components

Option 2: External Template (Recommended)

```
@Component({
  selector: 'app-user-list',
  templateUrl: './user-list.component.html', // Path to HTML file
  styleUrls: ['./user-list.component.css']   // Path to CSS file
})
export class UserListComponent {
  users = ['Alice', 'Bob', 'Charlie'];
}
```


Output:

When to use external template:

- Template is longer than 5 lines (most cases)
- Better code organization and readability
- IDE support for HTML syntax highlighting
- Easier to maintain and test

3.2.3 styles vs styleUrls

Similar to templates, you can define styles inline or in external files.

Inline Styles:

```
@Component({
  selector: 'app-badge',
  template: `<span class="badge">{{ text }}</span>`,
  styles: [
    .badge {
      background-color: #ff5722;
      color: white;
      padding: 4px 8px;
      border-radius: 12px;
      font-size: 12px;
      font-weight: bold;
    }
  ]
})
export class BadgeComponent {
  text = 'New';
}
```

External Styles (Recommended):

```
@Component({
  selector: 'app-dashboard',
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.css']
})
export class DashboardComponent {
}
```

```
styleUrls: ['./dashboard.component.css']
})
export class DashboardComponent { }
```

 Note:

Multiple Style Files: You can include multiple style files by providing an array:

```
styleUrls: ['./component.css', './component-theme.css', './component-responsive.css']
```

3.2.4 Complete Metadata Example

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-product-card',           // How to use in HTML
  templateUrl: './product-card.component.html', // View
  styleUrls: ['./product-card.component.css'] // Styles
})
export class ProductCardComponent implements OnInit {
  // Component properties
  productName: string = 'Laptop';
  price: number = 999;
  inStock: boolean = true;
  imageUrl: string = 'assets/laptop.jpg';

  // Constructor
  constructor() {
    console.log('ProductCardComponent constructor called');
  }

  // Lifecycle hook
  ngOnInit(): void {
    console.log('ProductCardComponent initialized');
    this.loadProductData();
  }

  // Component methods
  loadProductData(): void {
    // Load product data from service
  }

  addToCart(): void {
```

```
    console.log(`${this.productName} added to cart`);  
  }  
  
  formatPrice(): string {  
    return `$$${this.price.toFixed(2)}`;  
  }  
}
```

Output:

Component Execution Flow:

1. Constructor is called first: "ProductCardComponent constructor called"
2. `ngOnInit()` is called next: "ProductCardComponent initialized"
3. `loadProductData()` executes
4. Component renders with initial data
5. User interactions trigger methods like `addToCart()`

4. Root Component Overview

4.1 What is the Root Component?

The root component is the first component that Angular loads when the application starts. By convention, it is named `AppComponent` and has the selector `app-root`. All other components in the application are children (direct or nested) of this root component.

```
// app.component.ts  
import { Component } from '@angular/core';  
  
@Component({  
  selector: 'app-root', // Root selector  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent {  
  title = 'my-angular-app';  
}
```

4.2 Application Bootstrap Process

Understanding how Angular starts with the root component:

```
// Step 1: index.html (Entry Point)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>MyAngularApp</title>
  <base href="/">
</head>
<body>
  <app-root></app-root> <!-- Root component placeholder -->
</body>
</html>
```

```
// Step 2: main.ts (Bootstrap File)
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic()
  .bootstrapModule(AppModule) // Load and bootstrap the root module
  .catch(err => console.error(err));
```

```
// Step 3: app.module.ts (Root Module)
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent // Root component declared here
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
```

```
bootstrap: [AppComponent] // AppComponent is bootstrapped
})
export class AppModule { }
```

Output:

Bootstrap Flow:

1. Browser loads `index.html` , sees `<app-root></app-root>`
2. Angular executes `main.ts` , bootstraps `AppModule`
3. `AppModule` declares and bootstraps `AppComponent`
4. Angular finds `<app-root>` in HTML and replaces it with `AppComponent` 's template
5. Application renders and becomes interactive

4.3 Component Tree Structure

```
// Component Hierarchy Example
AppComponent (Root)
├── HeaderComponent
│   ├── LogoComponent
│   └── NavigationComponent
│       ├── NavItemComponent
│       └── NavItemComponent
├── MainContentComponent
│   ├── SidebarComponent
│   │   ├── MenuComponent
│   │   └── UserProfileComponent
│   └── DashboardComponent
│       ├── WidgetComponent
│       ├── ChartComponent
│       └── TableComponent
└── FooterComponent
    ├── ContactInfoComponent
    └── SocialLinksComponent
```

Output:

Component Tree Concept: AppComponent is the root (trunk of tree). All other components branch out from it. Each component can contain other components, creating a hierarchical structure.

5. Creating Components

5.1 Method 1: Using Angular CLI (Recommended)

Angular CLI provides the fastest and most consistent way to create components.

Basic Command:

```
ng generate component user-profile

// Shorthand
ng g c user-profile
```

Output:

CLI Output:

```
CREATE src/app/user-profile/user-profile.component.css (0 bytes)
CREATE src/app/user-profile/user-profile.component.html (26 bytes)
CREATE src/app/user-profile/user-profile.component.spec.ts (625 bytes)
CREATE src/app/user-profile/user-profile.component.ts (297 bytes)
UPDATE src/app/app.module.ts (484 bytes)
```

What CLI Does Automatically:

1. Creates a new folder: `src/app/user-profile/`
2. Generates four files:
 - `user-profile.component.ts` - Component class
 - `user-profile.component.html` - Template
 - `user-profile.component.css` - Styles
 - `user-profile.component.spec.ts` - Unit tests

3. Imports the component in the module
4. Adds the component to the module's declarations array
5. Uses consistent naming conventions

CLI Options:

```
// Create component without test file
ng g c user-profile --skip-tests

// Create component with inline template
ng g c user-profile --inline-template

// Create component with inline styles
ng g c user-profile --inline-style

// Create component in a specific folder
ng g c dashboard/user-profile

// Create component without creating folder
ng g c user-profile --flat

// Combine options
ng g c user-profile --skip-tests --inline-template
```

Output:

Generated Component (ng g c user-profile):

```
// user-profile.component.ts (Generated by CLI)
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-user-profile',
  templateUrl: './user-profile.component.html',
  styleUrls: ['./user-profile.component.css']
})
export class UserProfileComponent implements OnInit {

  constructor() { }

  ngOnInit(): void {
```

```
}  
  
}
```

5.2 Method 2: Manual Creation

Creating a component manually helps understand the structure, but is more error-prone.

Step 1: Create the folder and files

```
src/app/  
└─ product-card/  
    │ product-card.component.ts  
    │ product-card.component.html  
    │ product-card.component.css  
    └─ product-card.component.spec.ts (optional)
```

Step 2: Create the TypeScript class

```
// product-card.component.ts  
import { Component, OnInit } from '@angular/core';  
  
@Component({  
  selector: 'app-product-card',  
  templateUrl: './product-card.component.html',  
  styleUrls: ['./product-card.component.css']  
})  
export class ProductCardComponent implements OnInit {  
  
  constructor() { }  
  
  ngOnInit(): void {  
  }  
  
}
```

Step 3: Create the template


```
<!-- product-card.component.html -->
<div class="product-card">
  <p>product-card works!</p>
</div>
```

Step 4: Create the stylesheet

```
/* product-card.component.css */
.product-card {
  border: 1px solid #ccc;
  padding: 10px;
}
```

Step 5: Register in Module (Critical Step)

```
// app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { ProductCardComponent } from './product-card/product-card.component'; // Import

@NgModule({
  declarations: [
    AppComponent,
    ProductCardComponent // Declare the component
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

⚡ Important:

Critical: If you forget to import and declare the component in the module, Angular will not recognize it and you'll get an error: `'app-product-card' is not a known element`

5.3 CLI vs Manual Comparison

Aspect	Angular CLI	Manual Creation
Speed	Instant (1 command)	Several minutes
File Creation	All 4 files automatically	Create each file manually
Module Registration	Automatic	Manual (easy to forget)
Naming Convention	Always consistent	Can be inconsistent
Imports	Correct imports added	Must add imports manually
Error Prone	Minimal	High (typos, missing steps)
Best For	All production projects	Learning Angular internals

6. Component Structure

6.1 Complete Component File Structure

```

src/app/user-dashboard/
├─ user-dashboard.component.ts    // Component logic (TypeScript)
├─ user-dashboard.component.html  // Component template (HTML)
├─ user-dashboard.component.css   // Component styles (CSS)
└─ user-dashboard.component.spec.ts // Component tests (Jasmine/Karma)
  
```

6.2 File Purposes and Interactions

6.2.1 .ts File (TypeScript Logic)

Purpose: Contains the component class with properties, methods, and lifecycle hooks.

```

// user-dashboard.component.ts
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-user-dashboard',
  
```

```
templateUrl: './user-dashboard.component.html',
styleUrls: ['./user-dashboard.component.css']
})
export class UserDashboardComponent implements OnInit {
  // Properties
  userName: string = 'Alice Johnson';
  totalOrders: number = 15;
  accountBalance: number = 1250.50;
  lastLoginDate: Date = new Date();
  isVerified: boolean = true;

  // Array of objects
  recentOrders = [
    { id: 1, product: 'Laptop', amount: 999 },
    { id: 2, product: 'Mouse', amount: 25 },
    { id: 3, product: 'Keyboard', amount: 75 }
  ];

  // Constructor
  constructor() {
    console.log('Component created');
  }

  // Lifecycle hook
  ngOnInit(): void {
    console.log('Component initialized');
    this.loadDashboardData();
  }

  // Methods
  loadDashboardData(): void {
    // Simulate loading data
    console.log('Loading dashboard data...');
  }

  refreshData(): void {
    console.log('Refreshing data...');
    this.loadDashboardData();
  }

  viewOrder(orderId: number): void {
    console.log(`Viewing order: ${orderId}`);
  }
}
```

Output:

TypeScript File Contains:

- Import statements
- @Component decorator with metadata
- Component class with properties and methods
- Lifecycle hooks (ngOnInit, ngOnDestroy, etc.)
- Business logic and data management

6.2.2 .html File (Template)

Purpose: Defines the component's view using HTML and Angular template syntax.

```
<!-- user-dashboard.component.html -->
<div class="dashboard-container">
  <header class="dashboard-header">
    <h1>Welcome, {{ userName }}!</h1>
    <button (click)="refreshData()" class="refresh-btn">
      Refresh Data
    </button>
  </header>

  <div class="dashboard-stats">
    <div class="stat-card">
      <h3>Total Orders</h3>
      <p class="stat-value">{{ totalOrders }}</p>
    </div>

    <div class="stat-card">
      <h3>Account Balance</h3>
      <p class="stat-value">${{ accountBalance }}</p>
    </div>

    <div class="stat-card">
      <h3>Account Status</h3>
      <p class="stat-value">{{ isVerified ? 'Verified' : 'Not Verified' }}</p>
    </div>
  </div>

  <section class="recent-orders">
    <h2>Recent Orders</h2>
    <div class="order-list">
```

```
<div *ngFor="let order of recentOrders" class="order-item">
  <span>Order #{{ order.id }}: {{ order.product }}</span>
  <span class="order-amount">${{ order.amount }}</span>
  <button (click)="viewOrder(order.id)">View Details</button>
</div>
</div>
</section>
</div>
```

Output:

Template Features Used:

- `{{ userName }}` - String interpolation (display data)
- `(click)="refreshData()"` - Event binding (handle clicks)
- `*ngFor="let order of recentOrders"` - Structural directive (loop)
- `{{ isVerified ? 'Verified' : 'Not Verified' }}` - Ternary expression
- HTML structure and semantic tags

6.2.3 .css File (Styles)

Purpose: Defines the component's appearance and styling.

```
/* user-dashboard.component.css */
.dashboard-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
  font-family: Arial, sans-serif;
}

.dashboard-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 30px;
  padding-bottom: 20px;
  border-bottom: 2px solid #e0e0e0;
}

.dashboard-header h1 {
  color: #333;
```

```
font-size: 28px;
margin: 0;
}

.refresh-btn {
  background-color: #4CAF50;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 4px;
  cursor: pointer;
  font-size: 14px;
}

.refresh-btn:hover {
  background-color: #45a049;
}

.dashboard-stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  margin-bottom: 30px;
}

.stat-card {
  background-color: #f5f5f5;
  border-radius: 8px;
  padding: 20px;
  text-align: center;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.stat-card h3 {
  color: #666;
  font-size: 16px;
  margin: 0 0 10px 0;
}

.stat-value {
  color: #333;
  font-size: 32px;
  font-weight: bold;
  margin: 0;
}

.recent-orders h2 {
```

```
color: #333;
margin-bottom: 15px;
}

.order-list {
  background-color: #fff;
  border: 1px solid #e0e0e0;
  border-radius: 8px;
}

.order-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px;
  border-bottom: 1px solid #e0e0e0;
}

.order-item:last-child {
  border-bottom: none;
}

.order-amount {
  font-weight: bold;
  color: #4CAF50;
}

.order-item button {
  background-color: #2196F3;
  color: white;
  border: none;
  padding: 6px 12px;
  border-radius: 4px;
  cursor: pointer;
  font-size: 12px;
}

.order-item button:hover {
  background-color: #0b7dda;
}
```

Output:

Styled Component Result: The dashboard has a clean, professional appearance with a header, stat cards in a grid layout, and a list of recent orders. All colors, spacing, and layouts are defined in the CSS file and apply only to this component.

6.2.4 .spec.ts File (Unit Tests)

Purpose: Contains unit tests for the component to ensure it works correctly.

```
// user-dashboard.component.spec.ts
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { UserDashboardComponent } from './user-dashboard.component';

describe('UserDashboardComponent', () => {
  let component: UserDashboardComponent;
  let fixture: ComponentFixture<UserDashboardComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [ UserDashboardComponent ]
    })
    .compileComponents();
  });

  beforeEach(() => {
    fixture = TestBed.createComponent(UserDashboardComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should have correct user name', () => {
    expect(component.userName).toBe('Alice Johnson');
  });

  it('should load dashboard data on init', () => {
    spyOn(component, 'loadDashboardData');
    component.ngOnInit();
    expect(component.loadDashboardData).toHaveBeenCalled();
  });
});
```



```
it('should refresh data when button clicked', () => {
  spyOn(component, 'loadDashboardData');
  component.refreshData();
  expect(component.loadDashboardData).toHaveBeenCalled();
});
});
```

Output:

Test Execution: Run `ng test` to execute unit tests

- All tests pass: ✓ 4 tests passed
- Component is verified to work correctly
- Tests ensure functionality doesn't break with future changes

6.3 How Files Work Together

// Data Flow Between Files

1. Component Class (.ts)
↓ Provides data and methods
2. Template (.html)
↓ Displays data using {{ }} and calls methods via (click)
3. Styles (.css)
↓ Makes it look good with colors, layouts, spacing
4. Tests (.spec.ts)
→ Verifies everything works correctly

Result: Complete, tested, styled component

⚡ Important:

Remember: All four files work together to create a complete component. The .ts file provides the logic, the .html file defines the view, the .css file styles the appearance, and the .spec.ts file ensures quality through testing.

7. Summary

In this section, we covered the fundamentals of Angular components:

- **Components are building blocks** - Self-contained units with structure (HTML), logic (TypeScript), and style (CSS)
- **@Component decorator** - Provides metadata including selector, template, and styles
- **Root component** - AppComponent is the entry point, all other components branch from it
- **CLI vs Manual** - Angular CLI is the recommended way to create components (fast, consistent, automatic)
- **Four-file structure** - .ts (logic), .html (view), .css (styles), .spec.ts (tests)
- **Component registration** - Components must be declared in a module to be recognized by Angular

 Tip:

Next Steps: Now that you understand component basics, the next section will cover nested components, parent-child relationships, and how components communicate with each other in the component tree.

This document is part of the Angular Development Course series. For questions or clarifications, please refer to the course materials or contact your instructor.